

Milestone 2 — Learned Reconstruction: Requirements and Current State

CSE480 Machine Vision · Ain Shams University · Mechatronics Engineering

This report maps every requirement of the Milestone 2 brief to the code that exists and to what has actually been measured. **Milestone 2 is substantially outstanding**, and this document says so requirement by requirement rather than presenting partial work as finished. Milestone 1 is reported separately in [milestone_1_report.md](#).

Every figure quoted here is reproducible from the commands in §9.

1. Summary

Dataset preparation	Done
Siamese CNN implemented	Done
Graph neural network implemented	Done
Siamese CNN trained	Partly — 500 epochs, one head of three
GNN trained	Partly — 500 epochs, one head of three
Same split for both models	Done — the GNN's edges are built from the Siamese's examples
Model scores passed to the assembly algorithm	Not met
Reconstructed image + position/orientation + quality from a model	Not met
Comparison against the classical method	Partly — pairwise ranking only
Unseen <i>testing</i> set, separate from validation	Does not exist

Rank-1 over the held-out block, all four measures scored on **identical** gated candidate sets by the single protocol in `ml/ranking.py`:

Measure	Rank-1	Top-3	Median rank
Siamese CNN, epoch 500 (final)	40.0%	70.0%	2
GNN, epoch 500 (final)	40.0%	70.0%	2
Classical MGC	55.0%	80.0%	1
GNN, epoch 225 (best validation loss)	60.0%	80.0%	1

Read that last row carefully. The GNN's best-validation checkpoint is the first learned measure here to score above MGC — but only 20 pairs are scored, so one pair is five percentage points, and 60% against 55% is **12 correct against 11**. That is not a statistically meaningful lead, and it is not the checkpoint either run keeps. Both models

end their 500th epoch *below* the classical measure. Nothing model-driven has therefore been wired into the solver.

2. The output contract every model must satisfy

"Each model must produce the reconstructed puzzle image together with the predicted position and orientation of every piece and a numerical measure of reconstruction quality." — Milestone 2 brief

This is already implemented for the classical path, and a learned measure substitutes into it without changing any of it. The three required outputs are produced in two stages.

2.1 The search returns data, not pixels

`src/solver.py:solve()` returns a `Reconstruction` object. No image is involved:

Field	Contents
<code>grid</code>	the inferred <code>(rows, cols)</code> — not supplied to the solver
<code>placements</code>	one <code>Placement</code> or <code>None</code> per cell, in row-major cell order
<code>cost</code>	the search objective of the arrangement returned
<code>quality</code>	the blind quality figures of §2.4
<code>diagnostics</code>	beam width, per-state fan-out, flat weight, cell order, gate penalty, typical seam cost, unfilled cells

Each `Placement` is `(index, piece_id, rotation)` — the piece's position in the supplied bag, its own identifier, and the counter-clockwise quarter turns applied to it. `piece_ids()` and `rotations()` return those as flat per-cell lists, which **is** the "predicted position and orientation of every piece" the brief asks for. `complete` is true only when every cell is filled.

2.2 The image is a second, separate step

`src/solver.py:render_grid()` composites the reconstruction into a real picture of the assembled puzzle — the actual piece pixels, not a diagram or a schematic of the grid:

```
variants = build_variants(pieces)
result    = solve(pieces, variants=variants)
image     = render_grid(variants, result)      # HxWx3 uint8, BGR
```

Pieces are laid one *body square* apart, which is exactly the pitch at which the canonical representation makes interlocking pieces meet, so each piece's tabs enter its neighbours' blanks in the rendered output. For the 5×7 puzzle the result is **760 × 1016 × 3**.

2.3 What each entry point writes to disk

Every entry point writes **both** an image and a JSON record:

Command	Image	JSON
<code>python3 main.py --pieces data/pieces</code>	<code>results/reconstructed_images/reconstruction.png</code>	<code>results/reconstructed_images/reconstruction.metrics.json</code>
<code>python3 -m scripts.solve_blind</code>	<code>results/reconstructed_images/blind_reconstruction.png</code>	<code>results/evaluation_results/blind_reconstruction.json</code>
<code>notebooks/full_project_demo.ipynb</code>	<code>results/reconstructed_images/notebook_reconstruction.png</code>	<code>results/evaluation_results/notebook_reconstruction.json</code>

The two command-line JSONs carry `grid_inferred`, `complete`, the search objective, `quality`, `diagnostics`, `runtime_seconds`, the output path, and — only when a ground-truth layout is available — an `accuracy` block. (The notebook writes a shorter record: `grid`, `runtime_seconds`, `quality`, `accuracy`.) `main.py` also writes the full placement list, one record per cell:

```
{"cell": 0, "piece_id": 35, "rotation": 3}
```

Ground truth is read only *after* the search finishes, purely to score it. A model-driven run must keep that discipline.

2.4 The numerical measure of reconstruction quality

The quality score requires **no ground truth**, which is what makes it usable on an unseen puzzle. It is the *best-buddy* fraction: the proportion of expected seams at which both sides prefer each other over every alternative in the puzzle. `solve()` returns the full set:

Key	Meaning	Value on the photographed puzzle
<code>quality_score</code>	mutually-best seams ÷ seams expected	0.172
<code>mutual_best_fraction</code>	mutually-best seams ÷ seams realised	0.172
<code>best_partner_fraction</code>	directed seams whose partner was that side's cheapest	0.319
<code>mean_seam_cost</code>	mean cost of the realised seams	23.40
<code>median_seam_cost</code>	median of the same	17.47
<code>seams_realised / seams_expected</code>	58 / 58	all cells filled
<code>seams_violating_the_shape_gate</code>	seams the tab/blank gate rejects	0

Best-buddy agreement is the standard blind proxy for jigsaw reconstruction quality, and it is informative here: of the 21 best-buddy pairs the classical measure finds, 17 are true neighbours — 81% precision.

2.5 A known defect in the saved image

`render_grid` renders the arrangement **as placed**. A 5×7 grid has exactly one symmetry its seams cannot distinguish — the whole puzzle turned through 180° — so `src/evaluation.py:blind_accuracy` scores both readings and quotes the better one, recording which it used:

```
"reported_orientation": "rotated 180 degrees"
```

That rotation is **not** applied to the PNG. On the current run the saved image therefore reads upside down: the HIWIN logo and the red 飛躍新30+ mark appear inverted, and rotating the file 180° makes them read correctly. The JSON records the discrepancy honestly, but the image and the reported reading disagree. This should be fixed before a model-driven run produces images the same way — the fix is to apply the reported rotation in the two CLIs, or in `render_grid` itself, and to assert the agreement in `tests/test_assembly.py`.

3. Milestone 2 §1 — Dataset preparation

"Generate positive and negative side-pair samples... The same dataset split must be used for both models."

`ml/dataset.py`. **Done**, with the two caveats below stated rather than hidden.

What one example is. Each side is represented by the colour strip beneath it: the artwork sampled at 16 depths inwards from the boundary and 96 points along it, giving a `(3, 16, 96)` float32 array in $0..1$. That is deliberately the same information the classical MGC measure reads, so a learned score can be compared against it fairly. Two mating sides run in opposite physical directions, so the second strip of every pair is reversed along its length; after that reversal column i of strip A meets column i of strip B for positive and negative pairs alike, so the convention carries no signal a model could exploit.

Positives are all 58 internal seams of the verified layout, emitted in both directions, because the solver asks the question both ways.

Negatives are drawn only from pairs the tab/blank gate would admit — 3,944 of them — because those are the only pairs the solver ever asks about. Scoring pairs the silhouette already rejects would inflate the numbers for free. Eight negatives are sampled per positive.

The split is by piece, not by seam. There is one puzzle. Splitting its 58 seams at random would put the same physical piece on both sides of the split, and a model could then recognise a piece it had already seen rather than judge whether two edges continue each other. A contiguous 2×4 block of cells (cells 24–27 and 31–34) is held out instead; a

pair is a training pair only when both pieces are outside the block, a validation pair only when both are inside it, and pairs straddling the boundary are dropped.

	positives	negatives
train	84	672
validation	20	160
dropped at the split boundary	12	—

Caveat 1 — the training set is tiny. 84 positives is what one puzzle can supply after an honest split. §8 says what to do about it.

Caveat 2 — no augmentation is applied. The brief permits rotation, noise, illumination, contrast and colour augmentation. None is implemented. Rotation in particular is nearly free here, since `rotate_piece` already exists and is used to build the solved poses.

4. Milestone 2 §2 — Model development

Two fundamentally different architectures are implemented, and they are genuinely different models rather than one model with different hyperparameters.

	Siamese CNN (<code>ml/siamese.py</code>)	Graph NN (<code>ml/gnn.py</code>)
Input	two side strips, (3, 16, 96) each	node features (N, 32), edge index (2, E), edge attributes (E, 16)
Body	shared encoder: Conv 3→24, ReLU, MaxPool 2, Conv 24→48, ReLU, adaptive average pool, Linear 48→64	3 × message-passing layers: an MLP over [x_src, x_dst, edge_attr] into a GRU cell update, hidden width 64
Pair/edge head	[z_a - z_b , z_a ⊙ z_b] → Linear 128→64 → ReLU → Linear 64→6	[x_src, x_dst, edge_attr] → Linear 144→64 → ReLU → Linear 64→6
Parameters	22,870 (0.09 MB fp32)	126,982 (0.51 MB fp32)
Checkpoint on disk	<code>results/siamese/siamese.pt</code>	<code>results/gnn/gnn.pt</code>

Both expose the four outputs the brief asks of a compatibility model:

Brief requires	Head	State
whether two sides are neighbours	<code>neighbor_logit</code>	trained in both models
a numerical compatibility score	<code>compatibility</code> (sigmoid)	random initialisation — must not be reported
the predicted matching sides	implied by ranking <code>neighbor_logit</code> over candidates	usable, via <code>ml/train_siamese.py:_rank_report</code>
the required relative orientation	<code>orientation_logits</code> (4 classes)	random initialisation — must not be reported

Being explicit: three of the four required outputs are not trained, in either model. Only the neighbour decision has ever seen a gradient.

How the GNN sees the same data

The brief requires the two models to be *fundamentally different*, and they are. The Siamese judges one pair in isolation. The GNN makes every piece side a **node** and every candidate pair a **directed edge**, so three rounds of message passing let a side's representation be shaped by all the other partners competing for it before the edge head commits.

The edges are built from the examples `ml/dataset.build_pairs` returns — the same held-out block, the same 84 positives, the same 672 sampled negatives — so "the same dataset split for both models" holds by construction rather than by coincidence, and `tests/test_ml.py` asserts it.

Graph	Nodes	Edges
training	90 sides	756 (84 positive, 672 negative)
validation	27 sides	180 (20 positive, 160 negative)
scoring, at evaluation time	32 sides	318 — every gated candidate in the block

Node features are 31 numbers per side: twelve band means and twelve band standard deviations of the colour strip, the mean inward colour gradient per channel, the side's relief, and a one-hot of its tab/blank/flat label. Edge features are 16: twelve per-band mean absolute colour differences across the seam, the two reliefs, the gate flag, and the overall mean absolute difference. **None of them is the classical compatibility score** — handing a learned measure the number it is being compared against would settle the comparison by construction.

5. Milestone 2 §3 — Model training

The brief requires nine specific facts per model. Both are trained under deliberately matched conditions — same split, same loss, same optimiser, same learning rate, same batch size, same epoch count — so that the architecture is the only thing that differs.

Required statement	Siamese CNN	Graph neural network
Architecture	\$4 above	\$4 above
Input representation	two (3, 16, 96) colour strips, second reversed	31-d node features per side, 16-d edge features per candidate pair
Output representation	one logit per pair; positive means "adjacent"	one logit per edge; positive means "adjacent"
Loss function	BCEWithLogitsLoss, pos_weight = negatives / positives = 8	identical
Optimiser and learning rate	Adam, lr = 1e-3	Adam, lr = 1e-3
Batch size	32 pairs	32 edges per step, message passing over the full 756-edge graph
Epochs	500 (300, then resumed for 200 more)	500
Data augmentation	none	none
Model selection	the last epoch is kept — this is a defect , see below	same, though the best-validation weights are now also saved

Training and validation results

```
python3 -m ml.train_siamese --epochs 300 # then:
python3 -m ml.train_siamese --epochs 200 --resume # 60.3 s on a CUDA device

epoch  1/500  train_loss 1.2259  valid_loss 1.1923  valid_acc 0.722  recall 0.80
epoch 25/500  train_loss 0.7922  valid_loss 0.7711  valid_acc 0.761  recall 0.80
epoch 223/500                valid_loss 0.5200  <- best validation loss
epoch 300/500  train_loss 0.2968  valid_loss 3.9882  valid_acc 0.900  recall 0.45
epoch 375/500  train_loss 0.5792  valid_loss 1.0618  valid_acc 0.800  recall 0.80
epoch 500/500  train_loss 0.1276  valid_loss 7.6119  valid_acc 0.900  recall 0.40
```

```
python3 -m ml.train_gnn --epochs 500 # 94.2 s on a CUDA device

epoch  1/500  train_loss 1.2361  valid_loss 1.2224  valid_acc 0.733  recall 0.40
epoch 100/500  train_loss 0.8905  valid_loss 0.7265  valid_acc 0.750  recall 0.90
epoch 200/500  train_loss 0.0320  valid_loss 5.7742  valid_acc 0.861  recall 0.15
epoch 225/500  train_loss 0.6012  valid_loss 0.6619  valid_acc 0.783  recall 0.80  <- best
epoch 300/500  train_loss 0.2125  valid_loss 1.1696  valid_acc 0.833  recall 0.55
epoch 500/500  train_loss 0.0006  valid_loss 8.8430  valid_acc 0.861  recall 0.20
```

Both runs overfit, and the monitoring shows it clearly. Each reaches its best validation loss around epoch 223–225 and then degrades by an order of magnitude while training loss collapses towards zero — the GNN's reaches 0.0006 by epoch 500, which is memorisation of 756 edges, not learning. Validation *accuracy* climbs over the same stretch, which is exactly the trap the class weighting was meant to expose: with eight negatives per positive, accuracy rises while recall on true seams falls from 0.80–0.90 to 0.40 and 0.20. **Recall and loss are the figures to read here, not accuracy.**

The extra 200 epochs asked of the Siamese did not rescue it. They did raise its rank-1 from 30.0% to 40.0%, but its validation loss ended at 7.61 against 0.52 at epoch 223, and the trajectory in between is noise rather than progress — 1.06 at epoch 375, 5.08 at epoch 400, 1.20 at 425.

Two conclusions follow, and they are the actionable ones:

1. **Keeping the last epoch rather than the best is costing more than the models are gaining.** The GNN makes the point exactly: its epoch-225 weights rank 60.0% and its epoch-500 weights 40.0%. `ml/train_gnn.py` and `ml/train_siamese.py` now save the best-validation state alongside the final one, but neither *selects* it — early stopping is still the change to make.
2. **84 training positives is too few**, for a 22,870-parameter model and a 126,918-parameter one alike. More data is the real fix, not a smaller model.

The comparison that matters

Rank-1 over the held-out block, every measure scored on **identical** gated candidate sets by `ml/ranking.py`:

Measure	Rank-1	Top-3	Median rank	Pairs scored
Siamese CNN, epoch 500	40.0%	70.0%	2	20
GNN, epoch 500	40.0%	70.0%	2	20
Classical MGC	55.0%	80.0%	1	20
GNN, epoch 225 (best validation)	60.0%	80.0%	1	20

Three honest qualifications, all of which matter more than the ordering:

- **20 pairs is a very small sample.** One pair is five percentage points. The GNN's 60% against MGC's 55% is 12 correct against 11 — well inside noise.
- **The Siamese has no best-validation row.** Its best epoch (223) predates the addition of best-state tracking, so those weights were never saved. Only its final-epoch figure exists, while the GNN has both. Re-running the Siamese from scratch for 500 epochs would fill that cell in; it has not been done.
- These figures are over the held-out block only and are **not** comparable to the 46.6% quoted in the Milestone 1 report, which ranks against all 140 sides of the puzzle rather than the 32 sides of the block. The classical baseline is recomputed on the identical restricted candidate set so that the rows here can be compared with each other.

`python3 -m ml.evaluate_models` restates all of this into `results/evaluation_results/model_comparison.json`, and refuses to write the file if the two reports disagree about the split or about the classical baseline.

No testing set exists

The brief requires the final evaluation to be performed on an unseen **testing** set, separate from validation. Only a train/validation split exists, and the held-out block serves as both. Every number above is therefore a validation number, and it is reported as such.

6. Milestone 2 §4 — Puzzle assembly from model scores

Not done. No model score has ever reached the assembly algorithm.

The interface is ready and unchanged from Milestone 1. `src/solver.py` takes all its pairwise costs from two tensors built by

```
src.edge_compatibility.compatibility_matrices(variants), shaped (pieces, 4, pieces, 4)
```

for horizontal and vertical seams. Substituting a learned measure means filling those two tensors from the model instead of from MGC; everything downstream — the search, the once-per-piece constraint, the border handling, the rendering of §2 — is untouched.

That substitution is deliberately **not** made yet, because the learned measure currently ranks true partners worse than the classical one (§5), and Milestone 1 established by measurement that the pairwise measure, not the search, is what caps reconstruction accuracy. Wiring in a weaker measure would lower the 68.6% position accuracy, not raise it.

7. Milestone 2 §5 — Performance evaluation

The brief lists six axes of comparison. Filled in with what has actually been measured; blanks are marked, not guessed.

Axis	Classical (Milestone 1)	Siamese CNN	GNN
Matching / neighbour accuracy	34 / 58 seams (58.6%); rank-1 46.6% over all 140 sides, 55.0% on the block	rank-1 40.0% on the block at epoch 500	rank-1 40.0% at epoch 500; 60.0% at its best-validation epoch
Piece position accuracy	24 / 35 (68.6%)	not measured — never assembled	not measured — never assembled
Piece orientation accuracy	23 / 35 (65.7%)	not measured — orientation head untrained	not measured — orientation head untrained
Complete reconstruction	No	not measured	not measured
Reconstruction-quality score	0.172 best-buddy	not measured	not measured
Execution time	32-52 s per blind solve at beam 10,000	inference negligible; 60.3 s for the last 200 of 500 epochs	inference negligible; 94.2 s for 500 epochs
Model size / memory	no parameters; two (35,4,35,4) cost tensors	22,870 parameters, 0.09 MB fp32	126,918 parameters, 0.51 MB fp32

The most accurate method is still the classical one at the checkpoints either model actually keeps, and it is also the one that has ever produced a reconstruction. The single learned figure that exceeds it — the GNN's best-validation 60.0% — rests on one pair out of twenty and on a checkpoint the run does not select. It is a reason to add early stopping and more data, not a result to report as a win.

Between the two models, the GNN is the more promising: same split, same budget, same loss, and it is the only one whose weights ever ranked above MGC. It is also 5.5× larger and trains 1.6× slower, both of which are negligible at this scale.

How performance changes with puzzle size is not measured. The brief asks for it and it is answerable — `src/synthetic.py` cuts puzzles of any size from any image with the true adjacencies known by construction — but no such sweep has been run.

8. What has to happen next, in order

1. **Grow the training set.** `src/synthetic.py` cuts real interlocking puzzles of any size from any image, with every true adjacency known by construction. Thousands of positives are available this way; the held-out evaluation on the real photographed puzzle stays honest because the real puzzle is never trained on. This is the single change most likely to move the numbers.
2. **Keep the best epoch, not the last.** Early stopping on validation loss.
3. **Carve a real testing set**, separate from validation, so the final numbers answer the brief's question rather than a validation question.
4. **Train the orientation head**, which the brief explicitly requires, and only then report it.

5. **Re-run the Siamese from scratch for 500 epochs** so its best-validation checkpoint exists and the comparison table has no blank cell.
 6. **Only then substitute the learned measure** into `compatibility_matrices`, and only if it beats MGC on the held-out block. Re-run §7's table end to end once it does.
 7. **Fix the 180° image defect of §2.5** before model-driven runs start emitting images the same way.
-

9. Reproducing every figure in this report

```
python3 -m ml.train_siamese --epochs 300           # §5, ~78 s
python3 -m ml.train_siamese --epochs 200 --resume   # §5, to 500 total, ~60 s
python3 -m ml.train_gnn --epochs 500               # §5, ~94 s
python3 -m ml.evaluate_models                       # §5 comparison, instant
python3 -m scripts.solve_blind                      # §2 outputs: image + JSON, ~32 s
python3 main.py --pieces data/pieces                 # §2 outputs, with placements
python3 -m pytest -q tests/test_ml.py               # 12 data and wiring tests
```

Implementation: `ml/dataset.py` (pairs, graphs, features), `ml/ranking.py` (the one evaluation protocol every measure is scored by), `ml/siamese.py`, `ml/gnn.py`, `ml/train_siamese.py`, `ml/train_gnn.py`, `ml/evaluate_models.py`; the assembly and rendering the models must plug into are `src/solver.py` and `src/edge_compatibility.py`.